

Lecture 3: Ownership

Memory model, ownership

Willem Vanhulle

DevLab Rust 2025

Tuesday November 18, 2025

github.com/wvhulle/rust-course-ghent

1. Review	1
1.1. Installation	2
1.2. Exercises	3
2. Standard library traits	4
3. Ownership	15



1.1. Installation

Install and run Rustlings:

- `cargo install rustlings`
- `rustlings init` (creates new directory in the current directory)
- `cd rustlings/`
- open the rustlings folder in your IDE
- run rustlings in a background terminal
- follow the instructions in the terminal

(See <https://rustlings.rust-lang.org/setup/> for more.)

1.2. Exercises

1.2.1. Rustling exercises

Review first session:

- Ex 2. Functions
- Ex 4. Primitive types
- Ex 7. Structs
- Ex 8. Enums

Review last session:

- Ex 12. Option
- Ex 15. Traits
- Ex 5. Vectors
- Ex 9. Strings

If you have time:

- Ex 14. Generics
- Ex 11. HashMaps

1.2.2. Google exercises

- `session-3/examples/s2d6-closures.rs`
- `session-3/examples/s2d8-counter.rs`

1.2.3. Additional exercises

- `session-3/examples/s3e2-trait-conflicts.rs`
- `session-3/examples/s3e3-basic-traits.rs`
- `session-3/examples/s2d5-fn-trait.rs`
- `session-3/examples/s2d6-fnmut-trait.rs`
- `session-3/examples/s3e4-supertraits.rs`
- `session-3/examples/s3e5-multiple-bounds.rs`
- `session-3/examples/s3e6-dispatch.rs`

1. Review	1
2. Standard library traits	4
2.1. Comparison	5
2.2. Casting	9
2.3. Default	14
3. Ownership	15

```
1 struct Key {  
2     id: u32,  
3     metadata: Option<String>,  
4 }  
5 impl PartialEq for Key {  
6     fn eq(&self, other: &Self) -> bool {  
7         self.id == other.id  
8     }  
9 }
```

(playground link)

Can equality be implemented between different types?

2.1. Comparison

```
1 struct Key {  
2     id: u32,  
3     metadata: Option<String>,  
4 }  
5 impl PartialEq for Key {  
6     fn eq(&self, other: &Self) -> bool {  
7         self.id == other.id  
8     }  
9 }
```

(playground link)

Can equality be implemented between different types?

No, both types must be the same or one must implement `PartialEq` for the other type.

2.1. Comparison

```
1 use std::cmp::Ordering;
2 #[derive(Eq, PartialEq)]
3 struct Citation {
4     author: String,
5     year: u32,
6 }
7 impl PartialOrd for Citation {
8     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
9         match self.author.partial_cmp(&other.author) {
10             Some(Ordering::Equal) => self.year.partial_cmp(&other.year),
11             author_ord => author_ord,
12         }
13     }
14 }
```

(playground link)

When two references (not pointers) point to an equal object, are they equal?

When two references (not pointers) point to an equal object, are they equal?

Yes, references implement `PartialEq` by dereferencing both sides and comparing the underlying values.

```
1 fn main() {  
2     let a = "Hello";  
3     let b = String::from("Hello");  
4     assert_eq!(a, b);  
5 }
```

(playground link)

```
1  #[derive(Debug, Copy, Clone)]
2  struct Point {
3      x: i32,
4      y: i32,
5  }
6
7  impl std::ops::Add for Point {
8      type Output = Self;
9
10     fn add(self, other: Self) -> Self {
11         Self { x: self.x + other.x, y: self.y + other.y }
12     }
13 }
```

(playground link)

What happens if you write `Not (!)` in front of something that is not a boolean?

2.1. Comparison

```
1  #[derive(Debug, Copy, Clone)]
2  struct Point {
3      x: i32,
4      y: i32,
5  }
6
7  impl std::ops::Add for Point {
8      type Output = Self;
9
10     fn add(self, other: Self) -> Self {
11         Self { x: self.x + other.x, y: self.y + other.y }
12     }
13 }
```

(playground link)

What happens if you write Not (!) in front of something that is not a boolean?

It flips each bit.

```
1 fn main() {  
2     let s = String::from("hello");  
3     let addr = std::net::Ipv4Addr::from([127, 0, 0, 1]);  
4     let one = i16::from(true);  
5     let bigger = i32::from(123_i16);  
6     println!("{s}, {addr}, {one}, {bigger}");  
7 }
```

(playground link)

When you have a From implementation, do you receive a blanket Into implementation?

2.1. Comparison

```
1 fn main() {  
2     let s = String::from("hello");  
3     let addr = std::net::Ipv4Addr::from([127, 0, 0, 1]);  
4     let one = i16::from(true);  
5     let bigger = i32::from(123_i16);  
6     println!("{s}, {addr}, {one}, {bigger}");  
7 }
```

(playground link)

When you have a From implementation, do you receive a blanket Into implementation?

Yes.

When you have an Into implementation, do you receive a blanket From implementation?

2.1. Comparison

```
1 fn main() {  
2     let s = String::from("hello");  
3     let addr = std::net::Ipv4Addr::from([127, 0, 0, 1]);  
4     let one = i16::from(true);  
5     let bigger = i32::from(123_i16);  
6     println!("{s}, {addr}, {one}, {bigger}");  
7 }
```

(playground link)

When you have a From implementation, do you receive a blanket Into implementation?

Yes.

When you have an Into implementation, do you receive a blanket From implementation?

No.

Warning

Always implement From for your own types.



```
1 fn main() {  
2     let value: i64 = 1000;  
3     println!("as u16: {}", value as u16);  
4     println!("as i16: {}", value as i16);  
5     println!("as u8: {}", value as u8);  
6 }
```

(playground link)

Warning

Prefer using From and Into over as casting whenever possible.

Which trait should I use when casting is fallible (may go wrong)?

```
1 fn main() {  
2     let value: i64 = 1000;  
3     println!("as u16: {}", value as u16);  
4     println!("as i16: {}", value as i16);  
5     println!("as u8: {}", value as u8);  
6 }
```

(playground link)

Warning

Prefer using From and Into over as casting whenever possible.

Which trait should I use when casting is fallible (may go wrong)?

Use the TryFrom and TryInto traits.

```
1 use std::io::{BufRead, BufReader, Read, Result};
2
3 fn count_lines<R: Read>(reader: R) -> usize {
4     let buf_reader = BufReader::new(reader);
5     buf_reader.lines().count()
6 }
7
8 fn main() -> Result<()> {
9     let slice: &[u8] = b"foo\nbar\nbaz\n";
10    println!("lines in slice: {}", count_lines(slice));
11
12    let file = std::fs::File::open(std::env::current_exe()??);
13    println!("lines in file: {}", count_lines(file));
14    Ok(())
15 }
```

(playground link)

```
1 use std::io::{Result, Write};
2
3 fn log<W: Write>(writer: &mut W, msg: &str) -> Result<()> {
4     writer.write_all(msg.as_bytes())?;
5     writer.write_all("\n".as_bytes())
6 }
7
8 fn main() -> Result<()> {
9     let mut buffer = Vec::new();
10    log(&mut buffer, "Hello")?;
11    log(&mut buffer, "World")?;
12    println!("Logged: {buffer:?}");
13    Ok(())
14 }
```

(playground link)



2.2. Casting

2. Standard library traits

2.2.1. Rustlings

- `23_conversions/`

2.2.2. Google

- Read trait: `session-3/tests/s3e1-rot.rs`

2.3. Default

```
1  #[derive(Debug, Default)]
2  struct Derived {
3      x: u32,
4      y: String,
5      z: Implemented,
6  }
7
8  #[derive(Debug)]
9  struct Implemented(String);
10
11 impl Default for Implemented {
12     fn default() -> Self { Self("John Smith".into()) }
13 }
14
15 fn main() {
16     let default_struct = Derived::default();
17     dbg!(default_struct);
18
19     let almost_default_struct =
20         Derived { y: "Y is set!".into(), ..Derived::default() };
21     dbg!(almost_default_struct);
22
23     let nothing: Option<Derived> = None;
24     dbg!(nothing.unwrap_or_default());
25 }
```

[\(playground link\)](#)

1. Review	1
2. Standard library traits	4
3. Ownership	15
3.1. Memory layout	16
3.2. Stack and heap	17
3.3. Memory challenges	19
3.4. Ownership	20
3.5. Move semantics	21
3.6. Clone	25
3.7. Exercise	28

```
1 fn main() {  
2     let s1 = String::from("Hello");  
3 }
```

(playground link)

What data does the s1 variable contain at runtime?

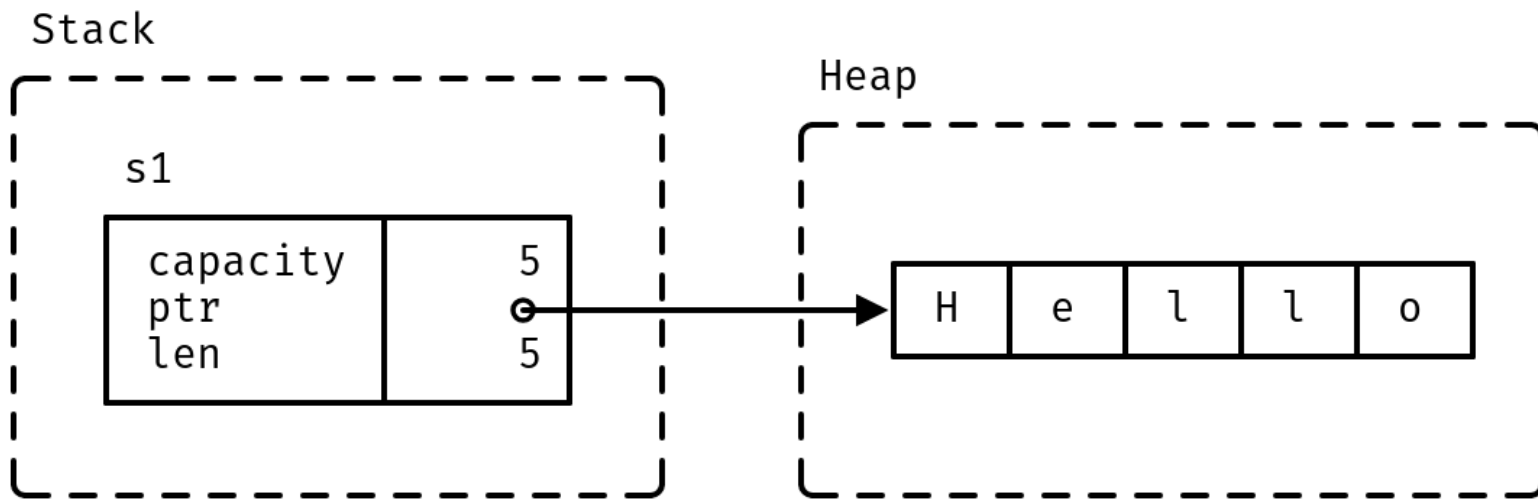
3.1. Memory layout

```
1 fn main() {  
2     let s1 = String::from("Hello");  
3 }
```

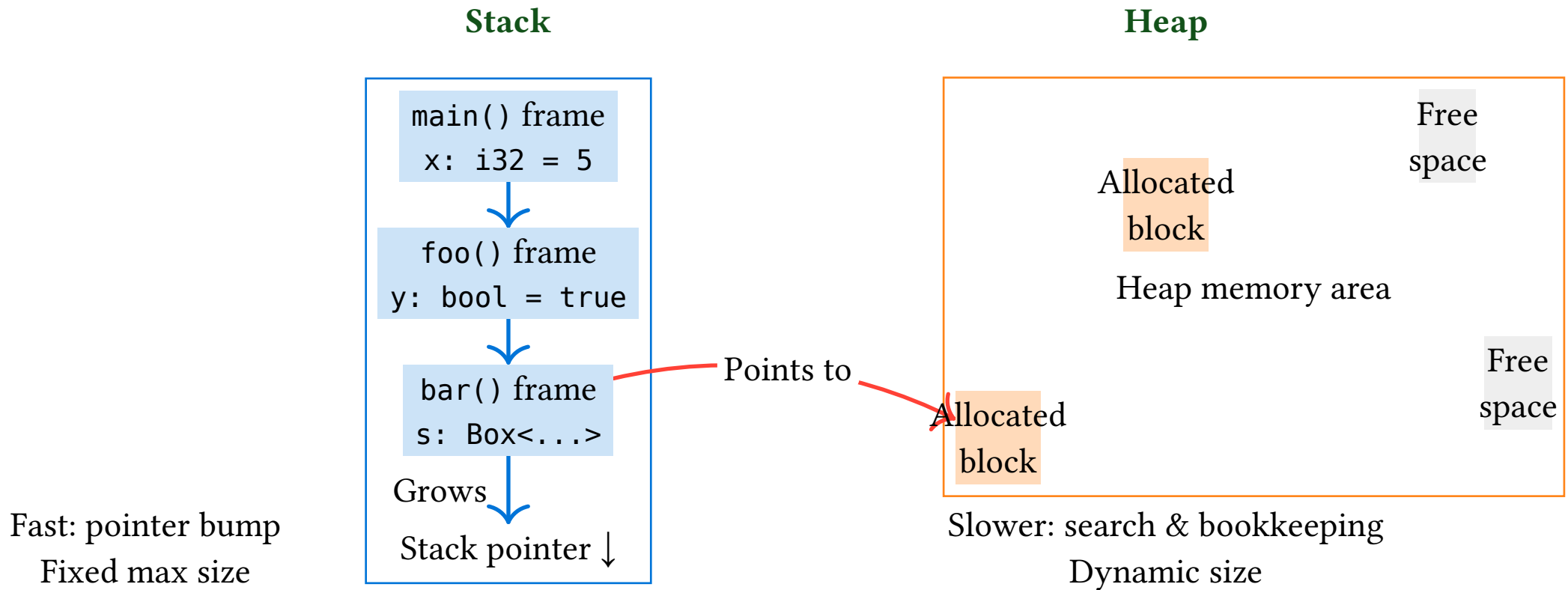
(playground link)

What data does the `s1` variable contain at runtime?

Capacity, address buffer, length.



3.2. Stack and heap



```
1  fn main() {
2      let mut s1 = String::from("Hello");
3      s1.push(' ');
4      s1.push_str("world");
5      // DON'T DO THIS AT HOME! For educational purposes only.
6      // String provides no guarantees about its layout, so this could lead to
7      // undefined behavior.
8      unsafe {
9          let (capacity, ptr, len): (usize, usize, usize) = std::mem::transmute(s1);
10         println!("capacity = {capacity}, ptr = {ptr:#x}, len = {len}");
11     }
12 }
```

(playground link)

3.3. Memory challenges

What is the most important challenge when managing memory?

3.3. Memory challenges

What is the most important challenge when managing memory?

Memory corruption.

What are common causes of memory corruption?

3.3. Memory challenges

What is the most important challenge when managing memory?

Memory corruption.

What are common causes of memory corruption?

Dangling pointers (use after free), double frees, buffer overflows.

What are the two different strategies to avoid memory corruption

3.3. Memory challenges

What is the most important challenge when managing memory?

Memory corruption.

What are common causes of memory corruption?

Dangling pointers (use after free), double frees, buffer overflows.

What are the two different strategies to avoid memory corruption

Automatic or manual memory management.



3.4. Ownership

Every Rust value has precisely one owner at all times.

```
1 struct Point(i32, i32);  
2  
3 fn main() {  
4     {  
5         let p = Point(3, 4);  
6         dbg!(p.0);  
7     }  
8     dbg!(p.1);  
9 }
```

(playground link)

What does this have in common with automatic garbage collection?

3.4. Ownership

Every Rust value has precisely one owner at all times.

```
1 struct Point(i32, i32);  
2  
3 fn main() {  
4     {  
5         let p = Point(3, 4);  
6         dbg!(p.0);  
7     }  
8     dbg!(p.1);  
9 }
```

(playground link)

What does this have in common with automatic garbage collection?

Rust's ownership system uses roots as well, but roots are determined statically at compile time.

```
1 fn main() {  
2     let s1 = String::from("Hello!");  
3     let s2 = s1;  
4     dbg!(s2);  
5     // dbg!(s1);  
6 }
```

(playground link)

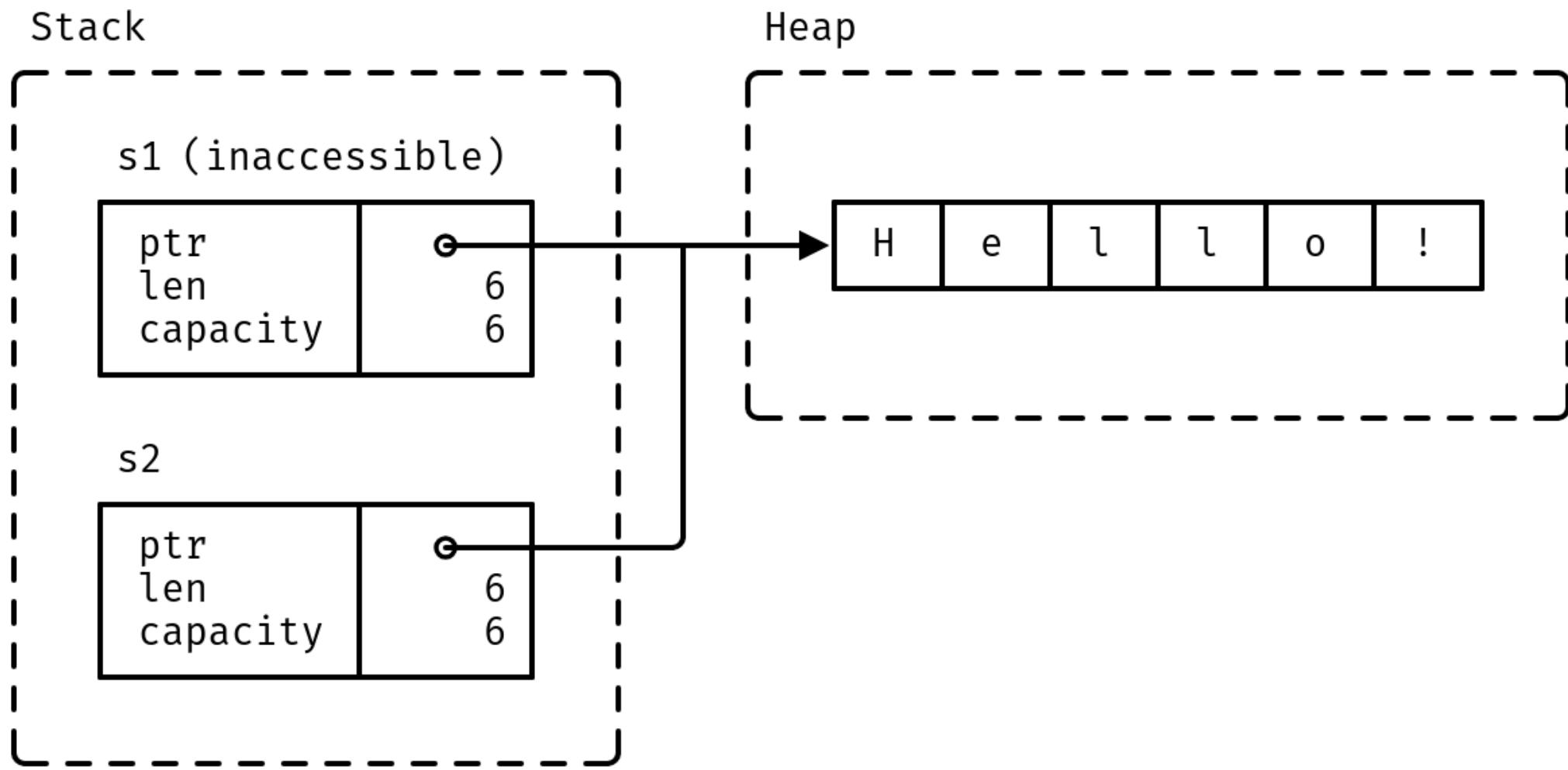
What is the ownership flow?

```
1 fn main() {  
2     let s1 = String::from("Hello!");  
3     let s2 = s1;  
4     dbg!(s2);  
5     // dbg!(s1);  
6 }
```

(playground link)

What is the ownership flow?

s1 owns the string. When s1 is assigned to s2, ownership moves to s2. s1 is no longer valid.



3.5. Move semantics

When you pass a value to a function, the value is assigned to the function parameter. This transfers ownership:

```
1 fn say_hello(name: String) {  
2     println!("Hello {name}")  
3 }  
4  
5 fn main() {  
6     let name = String::from("Alice");  
7     say_hello(name);  
8     // say_hello(name);  
9 }
```

(playground link)

How does this differ from what C++ does?

3.5. Move semantics

When you pass a value to a function, the value is assigned to the function parameter. This transfers ownership:

```
1 fn say_hello(name: String) {  
2     println!("Hello {name}")  
3 }  
4  
5 fn main() {  
6     let name = String::from("Alice");  
7     say_hello(name);  
8     // say_hello(name);  
9 }
```

(playground link)

How does this differ from what C++ does?

C++ copies the value by default, unless you use references or move semantics explicitly.

How do you restore C++ behaviour where you copy when necessary?

3.5. Move semantics

How do you restore C++ behaviour where you copy when necessary?

Derive the Copy trait.

```
1 fn main() {  
2     let x = 5;  
3     let y = x;  
4     println!("x = {x}, y = {y}");  
5 }
```

(playground link)

Are Strings in Rust Copy? (Do they get cloned automatically on assignment?)

3.5. Move semantics

How do you restore C++ behaviour where you copy when necessary?

Derive the Copy trait.

```
1 fn main() {  
2     let x = 5;  
3     let y = x;  
4     println!("x = {x}, y = {y}");  
5 }
```

(playground link)

Are Strings in Rust Copy? (Do they get cloned automatically on assignment?)

No.

```
1 fn say_hello(name: String) {  
2     println!("Hello {name}")  
3 }  
4  
5 fn main() {  
6     let name = String::from("Alice");  
7     say_hello(name.clone());  
8     say_hello(name);  
9 }
```

(playground link)

Values which implement Drop can specify code to run when they go out of scope:

```
1 struct Droppable {  
2     name: &'static str,  
3 }  
4  
5 impl Drop for Droppable {  
6     fn drop(&mut self) {  
7         println!("Dropping {}", self.name);  
8     }  
9 }
```

(playground link)

```

1  fn main() {
2      let a = Droppable { name: "a" };
3      {
4          let b = Droppable { name: "b" };
5          {
6              let c = Droppable { name: "c" };
7              let d = Droppable { name: "d" };
8              println!("Exiting innermost block");
9          }
10         println!("Exiting next block");
11     }
12     drop(a);
13     println!("Exiting main");
14 }

```

([playground link](#))

Explain what will be printed?

3.6. Clone

```
1  fn main() {  
2      let a = Droppable { name: "a" };  
3      {  
4          let b = Droppable { name: "b" };  
5          {  
6              let c = Droppable { name: "c" };  
7              let d = Droppable { name: "d" };  
8              println!("Exiting innermost block");  
9          }  
10         println!("Exiting next block");  
11     }  
12     drop(a);  
13     println!("Exiting main");  
14 }
```

(playground link)

Explain what will be printed?

...



3.7. Exercise

3.7.1. Google

- `session-3/examples/s3e4-builder.rs`

3.7.2. Rustlings

- `05_vecs/`
- `09_strings/`
- `06_move_semantics/`